**HCLTech Career Shaper Programme | Academic Year 2026-27**

**PROJECT SYNOPSIS**

# RAG-Based Document Q&A System

Retrieval-Augmented Generation over PDF and DOCX documents with LangChain, ChromaDB, local embeddings and Google Gemini

| Field | Details |
|---|---|
| Project Title | RAG-Based Document Q&A System (AI-Based QA for Policy Documents) |
| Project ID | P_121 |
| Domain | LLM / Python |
| Difficulty | Advanced |
| Format | Individual Project |
| Learner Name | Anand Choubey |
| Roll Number | 2300320120040 |
| College / Campus | ABES Engineering College, Ghaziabad |
| Mentor / Guide Name | Amit Bhatnagar |
| Academic Year | 2026-27 |
| Submission Date | 30 October 2026 |
| GitHub Repository | https://github.com/isitanand/rag-document-qa.git |

**Contents**

1. Problem Statement | 2. Proposed Solution | 3. System Architecture | 4. Technology Stack | 5. Dataset Used | 6. Key Results and Metrics | Application Output Screenshots | 7. Challenges and Resolutions | 8. References | 9. Ethical Compliance Declaration | Appendix

# 1. Problem Statement

| 5W1H | Project-specific response |
|---|---|
| Who? | Students, employees and analysts who work with large policy, research and project documents. |
| What? | Finding a specific answer inside long PDF/DOCX files is slow, and general chatbots answer from memory and may invent facts (hallucination) instead of the user's own documents. |
| When / where? | While studying, reviewing company policy or guideline documents, or checking research papers and project reports. |
| Why? | Fast, trustworthy answers with proof of the source save time and reduce the risk of acting on wrong information. |
| How? | Index documents into a vector database and answer questions only from the most relevant retrieved chunks, showing source citations. |

# 2. Proposed Solution and Approach

The project is a Retrieval-Augmented Generation (RAG) application built with Python and Streamlit. Users upload PDF or DOCX files, which are split into overlapping chunks, converted to vectors with a local HuggingFace model (all-MiniLM-L6-v2) and stored in a persistent ChromaDB collection. When a question is asked, the most similar chunks are retrieved and passed with the question to Google Gemini (or OpenAI) through LangChain. The model answers only from this context, refuses when the documents do not contain the answer, and every response lists its source file, chunk number and relevance score.

## 2.1 Main features

- **Document upload** for PDF and DOCX files, with a Process & Index action.
- **Smart chunking** with adjustable chunk size and overlap.
- **Local embeddings** (free, no API key needed for embeddings).
- **Persistent vector storage** in ChromaDB for fast similarity search.
- **Conversational Q&A** with chat history and a filter to search one, several or all documents.
- **Source citations** showing document, chunk text and relevance score for each answer.
- **Hallucination prevention**: answers are grounded in retrieved context only.
- **Dashboard and document manager** for documents, chunks, queries and storage, plus delete and clear options.

# 3. System Architecture

## 3.1 Data Flow Diagram - Level 0

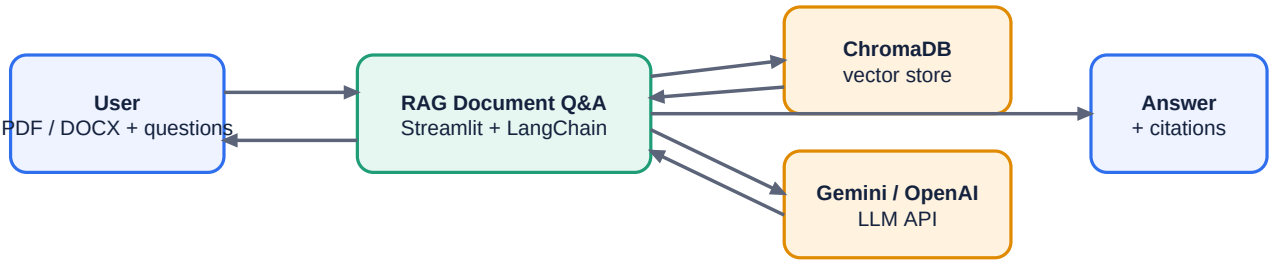



*Figure 1. Level 0 data flow: user documents and questions enter the RAG system, which uses ChromaDB and an LLM API to return cited answers.*

## 3.2 System block diagram

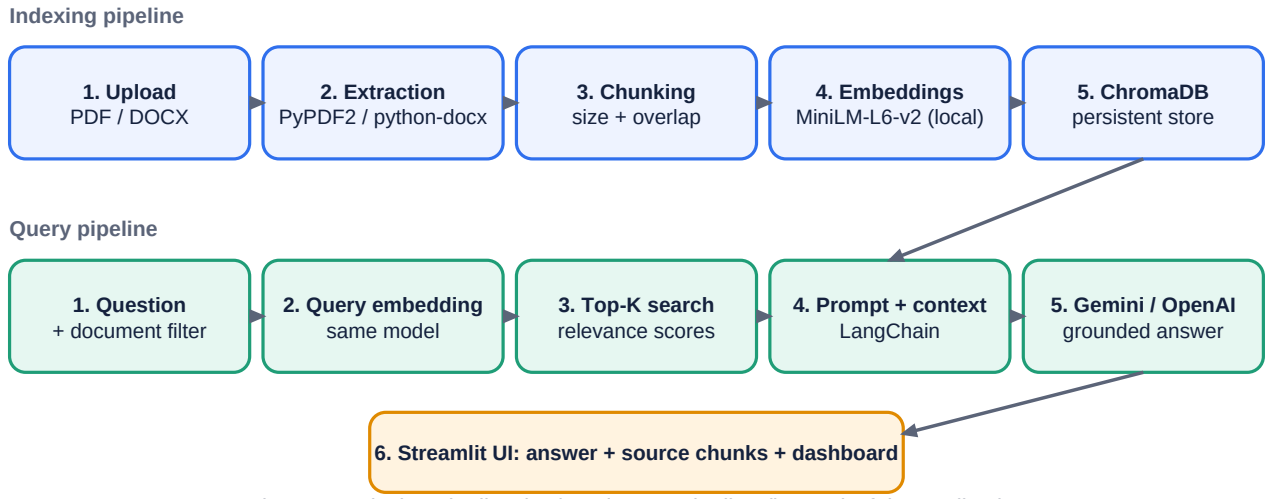



*Figure 2. Indexing pipeline (top) and query pipeline (bottom) of the application.*

## 3.3 Flow in brief

- Upload documents; text is extracted, chunked and embedded, then stored in ChromaDB.
- A question is embedded with the same model and the Top-K most similar chunks are retrieved.
- Chunks plus the question form the prompt; Gemini/OpenAI generates a grounded answer.
- The UI displays the answer, citations, relevance scores and updates dashboard statistics.

# 4. Technology Stack

| Layer | Technology | Role in this project |
|---|---|---|
| Frontend / UI | Streamlit | Chat, Documents and Dashboard pages, sliders for Top-K, chunk size and overlap. |
| Backend / Logic | Python 3.9+, LangChain | Pipeline orchestration, retrieval and prompt chain. |
| Database | ChromaDB (persistent) | Stores chunk embeddings and metadata; no separate DB server. |
| AI / ML | HuggingFace all-MiniLM-L6-v2; Google Gemini (default) / OpenAI | Local embeddings; answer generation. Retrieval-only mode works without an API key. |
| Document processing | PyPDF2, python-docx | Text extraction from PDF and DOCX. |

| Layer | Technology | Role in this project |
|---|---|---|
| Visualisation | Pandas, NumPy, Matplotlib, Seaborn, Streamlit charts | EDA plots and dashboard charts (chunks per document). |
| Hardware | Standard laptop (8 GB RAM), no GPU | Runs locally; internet only for the LLM API. |

### 4.1 Repository structure

```
rag-document-qa/
  app.py                    Streamlit entry point
  src/                      config, document_processor, text_splitter,
                            embeddings, vector_store, qa_chain, utils
  components/               sidebar, chat_interface, document_manager, dashboard
  eda/                      EDA script, cleaned data, visualizations
  dataset.csv, data_dictionary.csv, requirements.txt, .env.example
```

# 5. Dataset Used

| Field | Value |
|---|---|
| Dataset file name | dataset.csv (with data_dictionary.csv), provided in the project folder |
| Number of rows | 500 |
| Number of columns | 7 |
| Key columns | doc_id, doc_type, chunk_id, word_count, retrieval_score (also: query_type, answer_quality_score) |
| Documents indexed in the live demo | AgriFarm_Project_Diary_Anand_Choubey.docx (22 chunks) and research_paper1.pdf (75<br><br>testing |
| Additional data collected | None beyond the user-uploaded documents processed by the application |

The provided dataset describes document chunks and retrieval quality (document type, chunk size in words, retrieval score and answer quality) and was explored with the EDA script in the repository. The application itself answers from whichever documents the user uploads at run time.

# 6. Key Results and Metrics

| Metric | Observed value (from application screenshots) |
|---|---|
| Documents indexed / total chunks | 2 documents, 97 chunks (22 DOCX + 75 PDF); 3 documents, 136 chunks after adding a third file |
| Query response time | 3.4 seconds for a research-paper question (answer generated with 5 retrieved chunks) |
| Document processing time | 396 ms to process and index one DOCX file |
| Retrieval relevance | 66.2% on the top chunk for the AgriFarm question; 5 chunks retrieved per query |
| Storage used | 2.4 MB for the two-document knowledge base |
| Citation coverage | Every answer lists source file and chunk numbers, with expandable source text |

**Business value achieved:** users can query policy, research and project documents in natural language and get quick, source-backed answers instead of reading entire files, while the grounded prompt reduces unsupported answers.

# Application Output Screenshots

The following screens were captured from the running application (localhost) with the Google Gemini connection active.

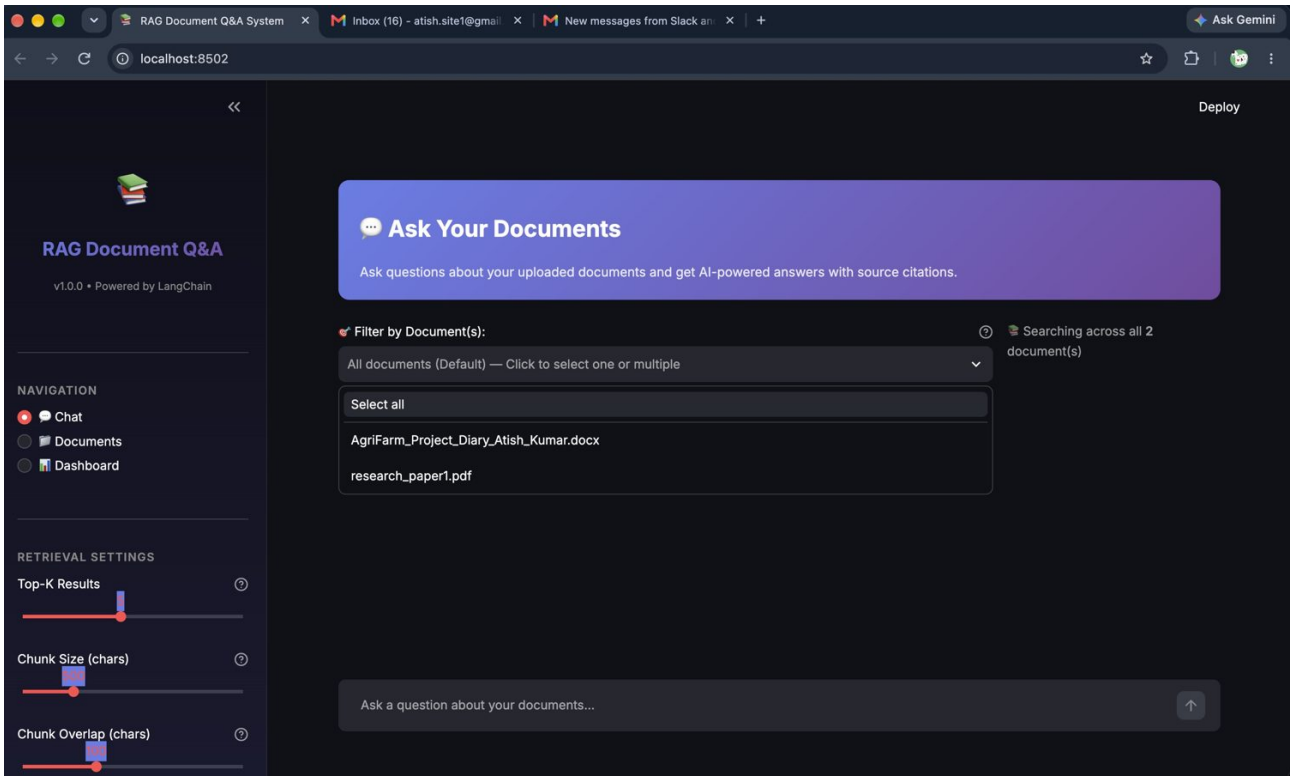



*Figure 3. Chat page with the document filter listing both indexed files (searching across all 2 documents).*

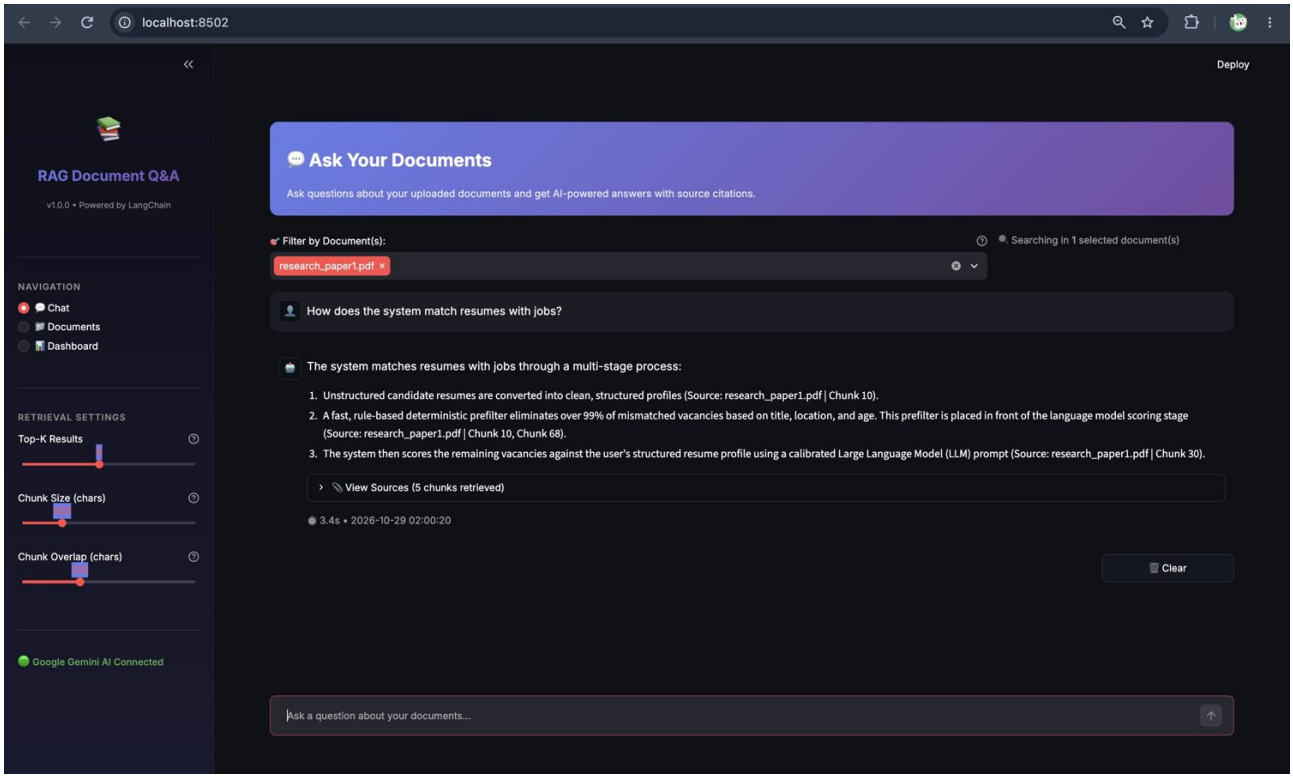



*Figure 4. Question on research_paper1.pdf: a numbered, cited answer with chunk references, 3.4 s response time and 5 retrieved chunks.*

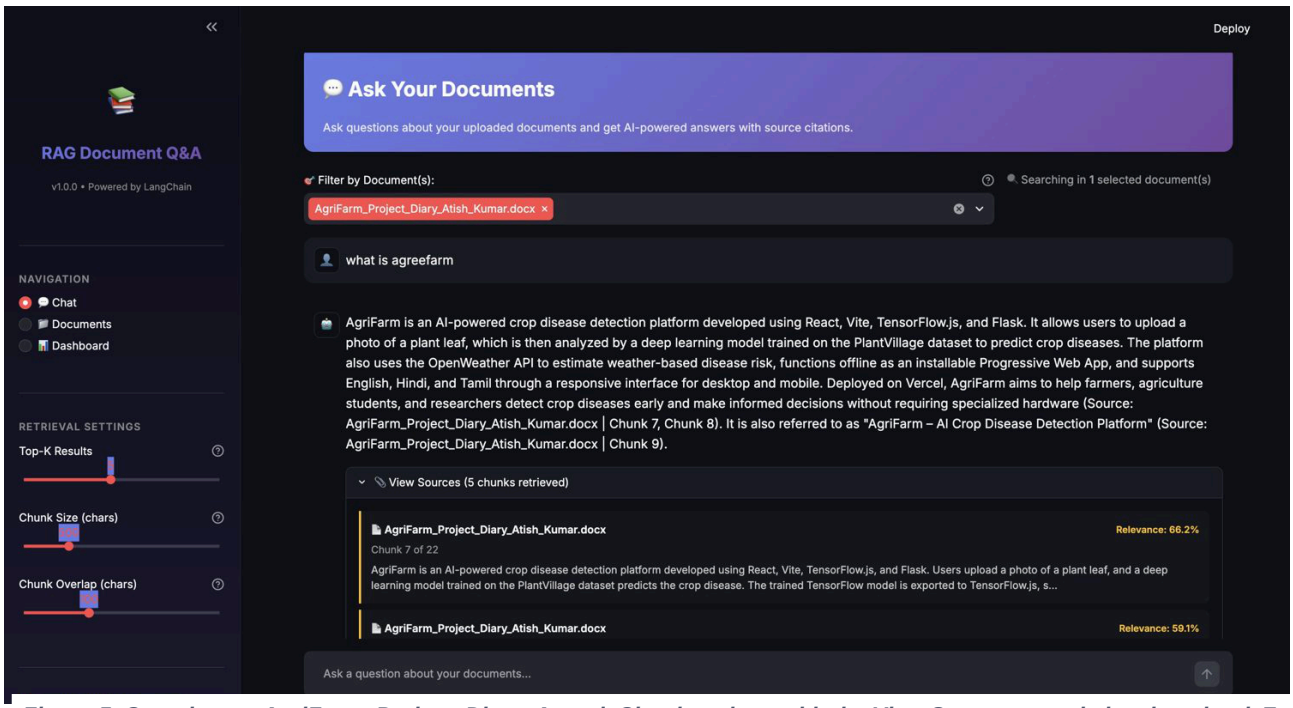



*Figure 5. Question on AgriFarm_Project_Diary_Anand_Choubey.docx with the View Sources panel showing chunk 7 of 22 and a*

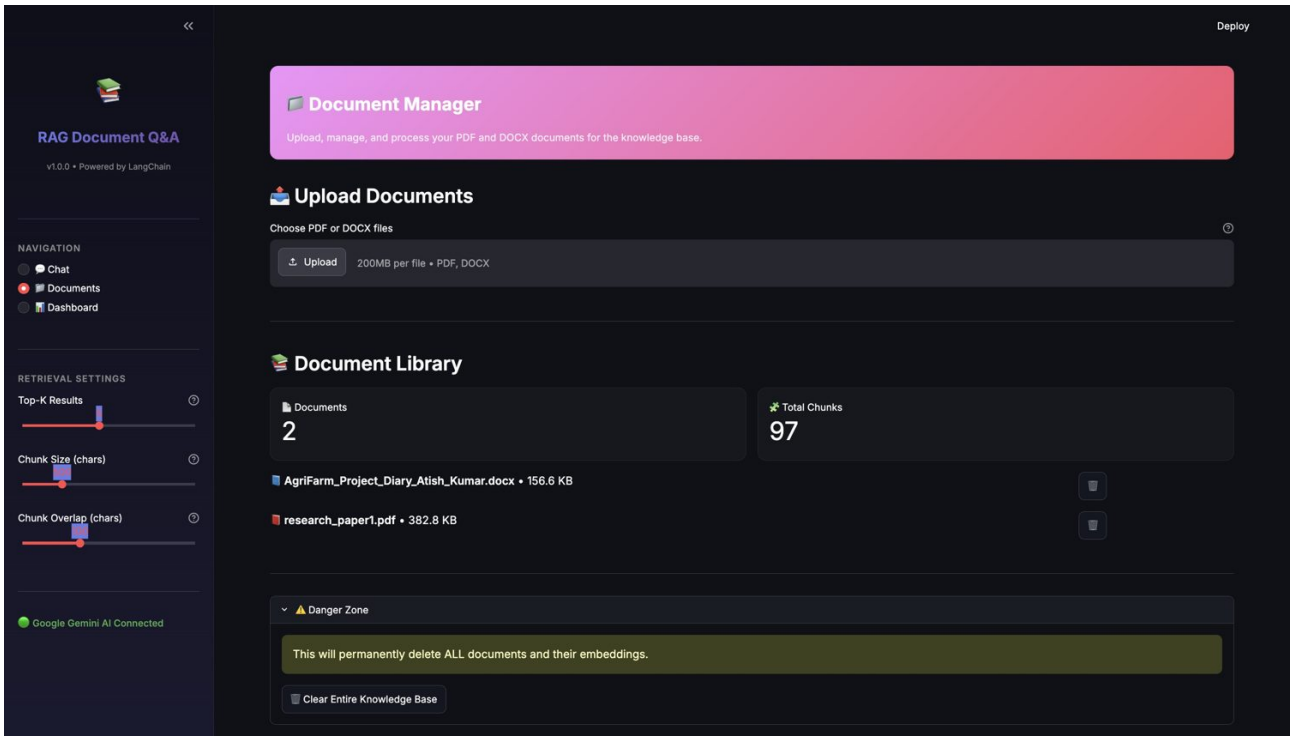



*Figure 6. Document Manager: PDF/DOCX uploader and Document Library (2 documents, 97 total chunks) with delete and Danger Zone controls.*

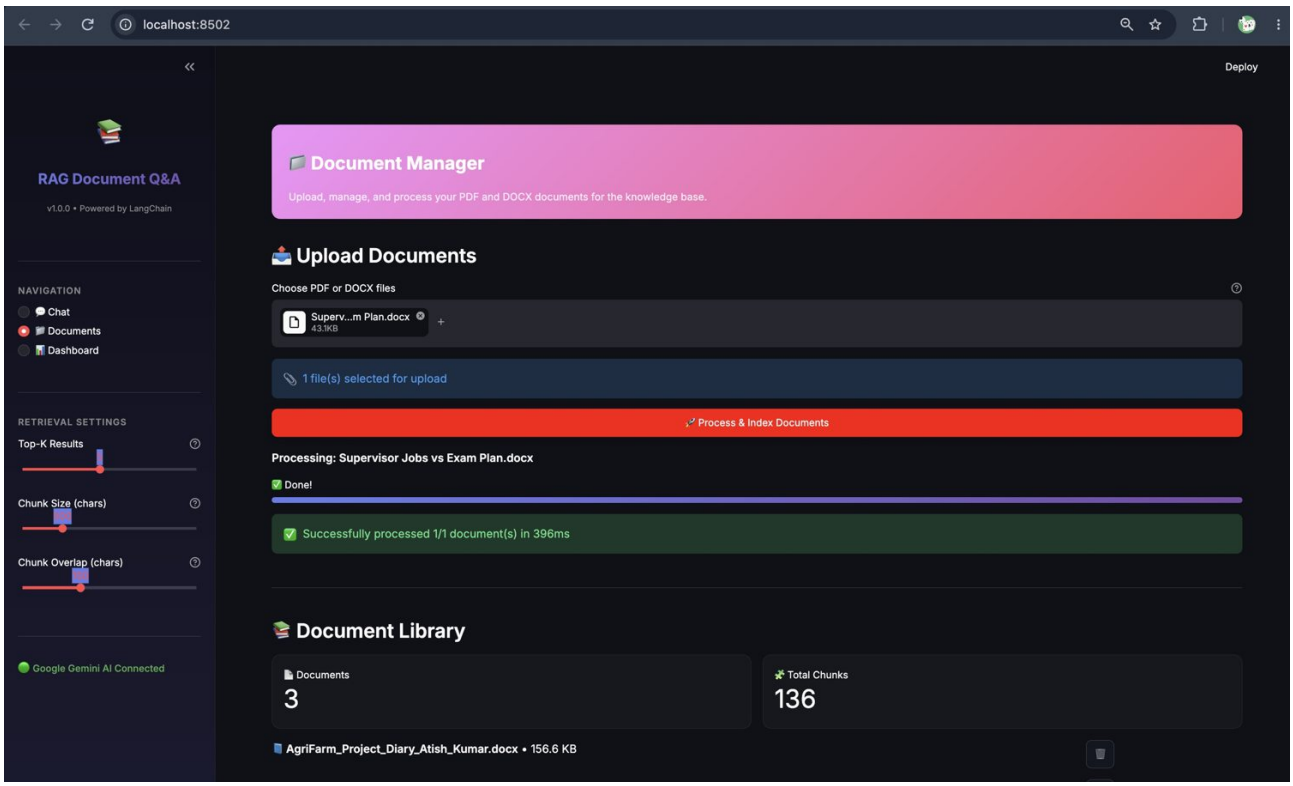



*Figure 7. Uploading Supervisor Jobs vs Exam Plan.docx: processed in 396 ms, library grows to 3 documents and 136 chunks.*

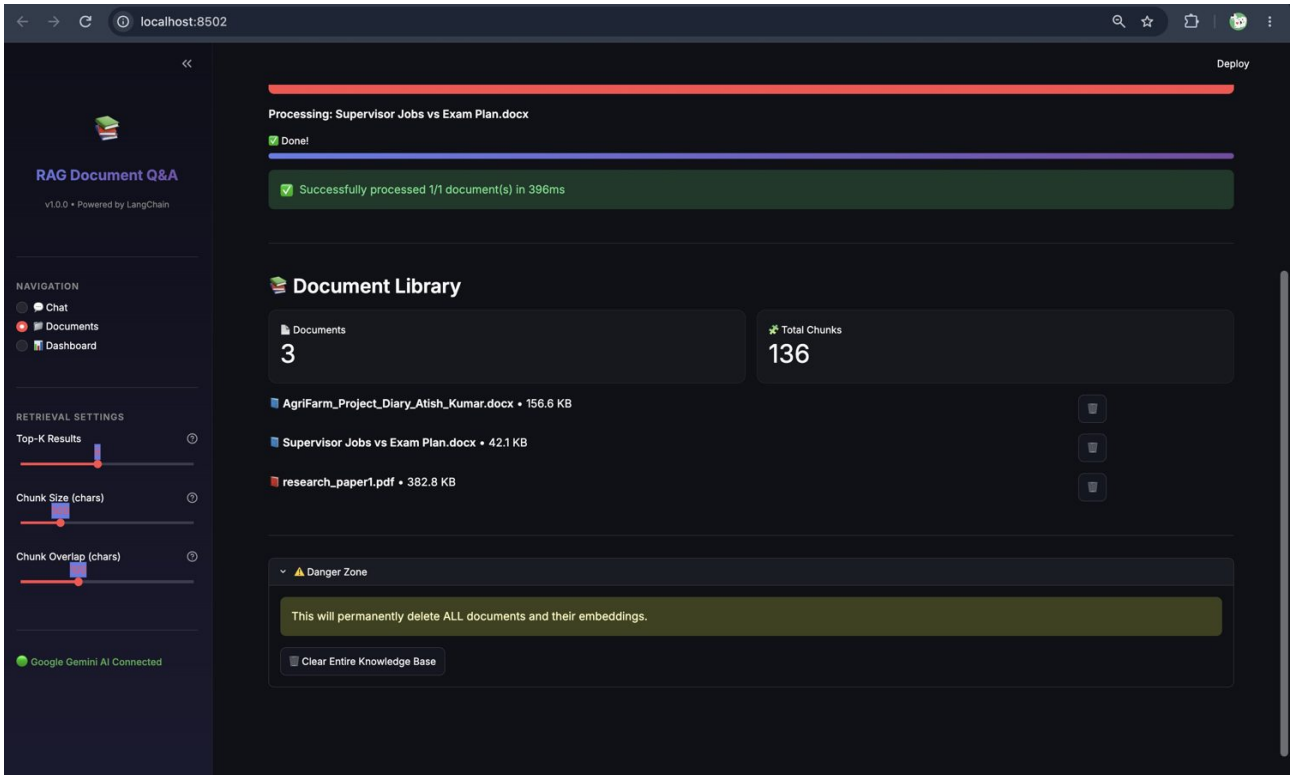



*Figure 8. Updated Document Library listing three documents with individual delete buttons.*

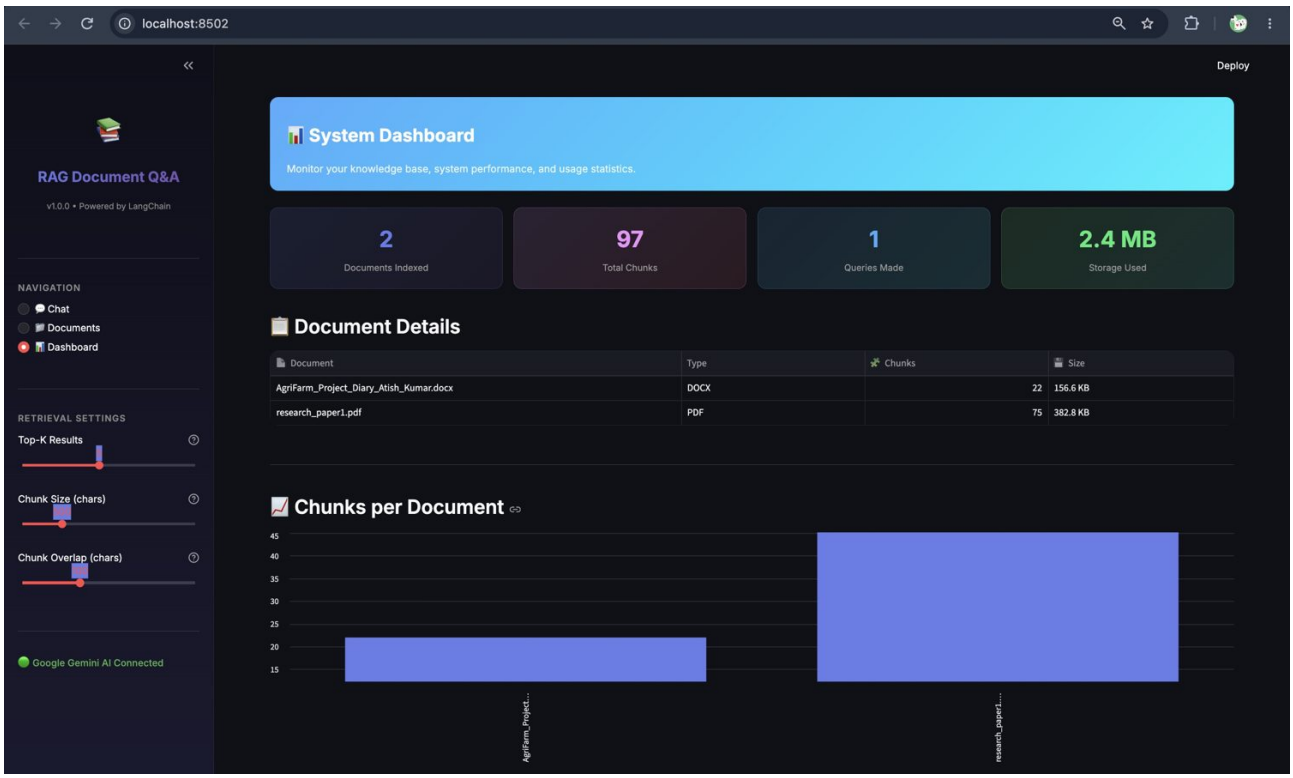



*Figure 9. System Dashboard: documents indexed, total chunks, queries made, storage used, document details and chunks per document.*

# 7. Challenges Faced and How Resolved

| Challenge | Resolution |
|---|---|
| The language model can hallucinate answers that are not in the documents. | Answers are generated only from retrieved chunks, the model is instructed to refuse unsupported questions, and source chunks with relevance scores are shown for verification. |
| Questions about one file were getting mixed with content from other files. | Added a multi-select document filter so a query can search one, several or all indexed documents. |
| Long documents cannot be sent to the model as a whole. | Documents are split into overlapping chunks (adjustable size and overlap) and only the Top-K most relevant chunks are sent. |
| Embedding APIs add cost and need keys. | Used a local HuggingFace sentence-transformer model; the app also works in retrieval-only mode without an LLM key. |
| Users need to trust and manage their knowledge base. | Built a Document Manager and Dashboard with per-file delete, clear-all and usage statistics. |

## 7.1 Known limitations

- Answer quality depends on the uploaded documents, chunk settings and the LLM used; scanned (image-only) PDFs are not covered.
- RAG reduces but cannot fully remove hallucination; outputs should not be used alone for legal or medical decisions.
- Reported metrics come from a small knowledge base (2 to 3 documents), not a large-scale benchmark.

## 7.2 Future improvements

- OCR for scanned documents and support for more file types.
- Quantitative evaluation (retrieval hit rate, answer faithfulness) on a labelled question set.
- User login with per-user knowledge bases and deployment to a cloud host.

# 8. References


- Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401. https://arxiv.org/abs/2005.11401
- LangChain Documentation. https://python.langchain.com
- Chroma Documentation. https://docs.trychroma.com
- Hugging Face. sentence-transformers/all-MiniLM-L6-v2 model card. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
- Streamlit Documentation. https://docs.streamlit.io
- Google AI Studio / Gemini API Documentation. https://ai.google.dev


# 9. Ethical Compliance Declaration

I confirm that:

[ ] All dataset usage complies with the source licence.

[ ] No personally identifiable information (PII) is stored without consent.

[ ] AI tools used for code/writing are disclosed in the Appendix.

[ ] This project complies with the DPDP Act 2023 where applicable.

Learner Signature: _____ Date: _____

# Appendix: AI Tools Disclosure

Google Gemini (via API) is used inside the application to generate answers; HuggingFace all-MiniLM-L6-v2 is used for embeddings. An AI assistant (Claude) was used to help draft and format this synopsis from the project's repository, template and screenshots.

# GitHub Repository

Source code: https://github.com/isitanand/rag-document-qa.git